

CM01 - Introduction, Linux, Shell

Course organization, OS model, paths, permissions, search, redirections

Louis Ledoux

2026-07-05

Table of contents

1 Course Organization	2
1.1 Main Path	3
1.2 Worked Example	3
1.3 Anecdote Or Motivation	4
1.4 Check Question	4
1.5 Backup Index	5
1.6 Backup: UE Outils et programmation en C L3 Informatique	7
1.7 Backup: \$whoami	8
1.8 Backup: Organisation 2025-2026	9
1.9 Backup: Introduction	10
1.10 Backup: Creation...From a text file to an executable	10
1.11 Backup: Layered view of computer organisation	11
1.12 Backup: L5: Problem-oriented language	12
1.13 Backup: Layered view of computer organisation	12
1.14 Backup: L4: Assembly Language	13
1.15 Backup: Layered view of computer organisation	13
1.16 Backup: L3: Operating system	14
1.17 Backup: Layered view of computer organisation	15
1.18 Backup: L2: Instruction Set Architecture	16
1.19 Backup: Layered view of computer organisation	17
1.20 Backup: L1: Micro architecture	18
1.21 Backup: Layered view of computer organisation	19
1.22 Backup: L0: Digital Logic	20
1.23 Backup: Layered view of computer organisation	21
1.24 Backup: Outline	23
2 Linux and OS	24
2.1 Main Path	24
2.2 Worked Example	25
2.3 Anecdote Or Motivation	25
2.4 Check Question	26
2.5 Backup Index	26
2.6 Backup: Part A: Linux and Shell scripting	29
2.7 Backup: Operating system	29
2.8 Backup: Linux	30
2.9 Backup: Some differences with Windows	31
2.10 Backup: Interface with the system: The Shell	32
2.11 Backup: Shell	32
3 Directories and Paths	33
3.1 Main Path	33

3.2	Worked Example	34
3.3	Anecdote Or Motivation	34
3.4	Check Question	35
3.5	Backup Index	35
3.6	Backup: Directories	37
3.7	Backup: Paths	38
3.8	Backup: Wildcard characters	39
3.9	Backup: How to treat them as normal (by the Shell)	40
3.10	Backup: Useful information	40
4	Commands and Files	41
4.1	Main Path	41
4.2	Worked Example	42
4.3	Anecdote Or Motivation	42
4.4	Check Question	43
4.5	Backup Index	43
4.6	Backup: Structure of a command	46
4.7	Backup: Shell Commands	47
4.8	Backup: Directory management	48
4.9	Backup: Listing	49
4.10	Backup: File management	50
4.11	Backup: File management	50
4.12	Backup: File management: Links	51
4.13	Backup: Directory/File management: Examples	52
5	Permissions and Search	52
5.1	Main Path	53
5.2	Worked Example	53
5.3	Anecdote Or Motivation	54
5.4	Check Question	54
5.5	Backup Index	55
5.6	Backup: Directory/File permissions	57
5.7	Backup: Directory/File permissions	59
5.8	Backup: Directory/File permissions: Examples	60
5.9	Backup: File management: Search files	60
5.10	Backup: File management: Search files	61
5.11	Backup: File management: Search files: Examples	62
5.12	Backup: File management: Search in files	63
6	Redirections	63
6.1	Main Path	63
6.2	Worked Example	64
6.3	Anecdote Or Motivation	64
6.4	Check Question	65
6.5	Backup Index	65
6.6	Backup: Redirections	67
6.7	Backup: Redirections: Examples	68
6.8	Backup: Redirections: Examples	69
6.9	Backup: Redirections: Examples	71

1 Course Organization

Live pacing: about 13 min

Question. What is this course really about, and how will students be evaluated?

Core claim. OPC is not only C syntax: it is the practice of turning text, tools, files, and memory into reliable programs.

Target capability. Students can explain the path from source file to executable and know where Unix, C, TD, and TP fit.

1.1 Main Path

- Start from the question: What is this course really about, and how will students be evaluated?
- Build the model: OPC is not only C syntax: it is the practice of turning text, tools, files, and memory into reliable programs.
- End with a checkable action: Students can explain the path from source file to executable and know where Unix, C, TD, and TP fit.
- Keep only this slide on the horizontal path during the live lecture.

UE Outilset programmation en C L3 Informatique

#1

1.2 Worked Example

- Use this as the live trace: Walk from `foo.c` to `./foo`, then ask what each tool has to know before the program can run.
- Representative source slide: 7
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

L5: Problem-oriented language

- File foo.c is written with the syntax of C language

```
#include <stdio.h>
int main()
{
    int i = 17;
    int j = 21;
    int k = i+j;
    printf("i+j=%d\n", k);
    return 0;
}
```

1,1 Top

#7

1.3 Anecdote Or Motivation

Use the first lecture to reset expectations: compilation errors and shell errors are feedback from tools, not random obstacles.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

1.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Outline

- Part 1: Linux and Shell scripting:
- 2 CM, 1 TD, 2 TP
- Part 2: C Language

- Kahoot time ! (2 Q)
- Goto: <https://kahoot.it/>
- Insert PIN number
- Insert a nickname

#19

1.5 Backup Index

Original slides 1-19

UE Outils et programmation en C L3 Informatique

#1

slide 1

\$whoami

- Name: Angeliki
- Surname: Kritikakou
- Email: angeliki.kritikakou@univ-rennes1.fr
- Position:
 - Associate Professor (HDR) - Researcher
 - Innovation Chair IUF
- Teaching: ISTIC mainly L3
- Research: TARAN team, INRIA-IRISA
- Research Interests:
 - Computer Architecture, Embedded Systems, Mixed-critical Systems, Fault Tolerance, Low energy design, Task Scheduling and Allocation, Memory Management, Design Space Exploration



#2

slide 2

Organisation 2025-2026

- CM: Angeliki Kritikakou
- 11 main courses
- Interactive course: Kahoot Tests
- TD: A. Mostefaoui, S. Errabii, F. Santos, C. Viho
- 3 exercise courses
- TP: A. Mostefaoui, S. Errabii, F. Santos, C. Viho
- 13 lab exercises
- Note: 1/2 Theoretical exam (15% CC-Unix, 35 % CC-C) + 1/2 Final Labexam (50%)
- Theoretical exam: Multiple choice questions
- Final lab exam:
- Independent
- 3 November 2025 (Last Lab)
- Duration: 1h (slot 15:00 or 16:05)

#3

slide 3

- Introduction

#4

slide 4

Creation...From a text file to an executable

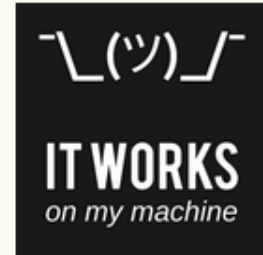


- ?

- foo.c

```
#include <stdio.h>
int main()
{
    int i = 17;
    int j = 21;
    int k = i+j;
    printf("i+j=%d\n", k);
    return 0;
}
```

1,1 Top



```
$ ./foo
i+j=38
$
```

#5

slide 5

Layered view of computer organisation

- Problem-oriented language level

#6

slide 6

Anchors: UE Outils et programmation en C L3 Informatique; \$whoami; Organisation 2025-2026; Introduction; Creation...From a text file to an executable; Layered view of computer organisation

1.6 Backup: UE Outils et programmation en C L3 Informatique

Original slide 1

1.7 Backup: \$whoami

Original slide 2

- Name: Angeliki
- Surname: Kritikakou
- Email: angeliki.kritikakou@univ-rennes1.fr
- Position:
- Associate Professor (HDR) - Researcher
- Innovation Chair IUF
- Teaching: ISTIC mainly L3
- Research: TARAN team, INRIA-IRISA
- Research Interests:
- Computer Architecture, Embedded Systems, Mixed-critical Systems, Fault Tolerance, Low energy design, Task Scheduling and Allocation, Memory Management, Design Space Exploration

\$whoami

- Name: Angeliki
- Surname: Kritikakou
- Email: angeliki.kritikakou@univ-rennes1.fr
- Position:
- Associate Professor (HDR) - Researcher
- Innovation Chair IUF
- Teaching: ISTIC mainly L3
- Research: TARAN team, INRIA-IRISA
- Research Interests:
- Computer Architecture, Embedded Systems, Mixed-critical Systems, Fault Tolerance, Low energy design, Task Scheduling and Allocation, Memory Management, Design Space Exploration



#2

1.8 Backup: Organisation 2025-2026

Original slide 3

- CM: Angeliki Kritikakou
- 11 main courses
- Interactive course: Kahoot Tests
- TD: A. Mostefaoui, S. Errabii, F. Santos, C. Viho
- 3 exercise courses
- TP: A. Mostefaoui, S. Errabii, F. Santos, C. Viho
- 13 lab exercises
- Note: 1/2 Theoretical exam (15% CC-Unix, 35 % CC-C) + 1/2 Final Lab exam (50%)
- Theoretical exam: Multiple choice questions
- Final lab exam:
- Independent
- 3 November 2025 (Last Lab)
- Duration: 1h (slot 15:00 or 16:05)

Organisation 2025-2026

- CM: Angeliki Kritikakou
- 11 main courses
- Interactive course: Kahoot Tests
- TD: A. Mostefaoui, S. Errabii, F. Santos, C. Viho
- 3 exercise courses
- TP: A. Mostefaoui, S. Errabii, F. Santos, C. Viho
- 13 lab exercises
- Note: 1/2 Theoretical exam (15% CC-Unix, 35 % CC-C) + 1/2 Final Labexam (50%)
- Theoretical exam: Multiple choice questions
- Final lab exam:
- Independent
- 3 November 2025 (Last Lab)
- Duration: 1h (slot 15:00 or 16:05)

#3

1.9 Backup: Introduction

Original slide 4

- Introduction

#4

1.10 Backup: Creation...From a text file to an executable

Original slide 5

- ?
- foo.c

```
$ ./foo
i+j=38
$
```

Creation...From a text file to an executable



-?

- foo.c

```
#include <stdio.h>
int main()
{
    int i = 17;
    int j = 21;
    int k = i+j;
    printf("i+j=%d\n", k);
    return 0;
}
```

1,1 Top



```
$. ./foo
i+j=38
$
```

#5

1.11 Backup: Layered view of computer organisation

Original slide 6

- Problem-oriented language level
- L5

Layered view of computer organisation

- Problem-oriented language level

#6

1.12 Backup: L5: Problem-oriented language

Original slide 7

- File foo.c is written with the syntax of C language

L5: Problem-oriented language

- File foo.c is written with the syntax of C language

```
#include <stdio.h>
int main()
{
    int i = 17;
    int j = 21;
    int k = i+j;
    printf("i+j=%d\n", k);
    return 0;
}
```

1,1 Top

#7

1.13 Backup: Layered view of computer organisation

Original slide 8

- Problem-oriented language level
- L5
- Translation
- (Compilation)
- Assembly language level
- L4

Layered view of computer organisation

- Problem-oriented language level

- Translation
- (Compilation)

- Assembly language level

#8

1.14 Backup: L4: Assembly Language

Original slide 9

- foo.s:
- Result of compilation

```
$ gcc -S foo.c
```

L4: Assembly Language

- foo.s:
- Result of compilation

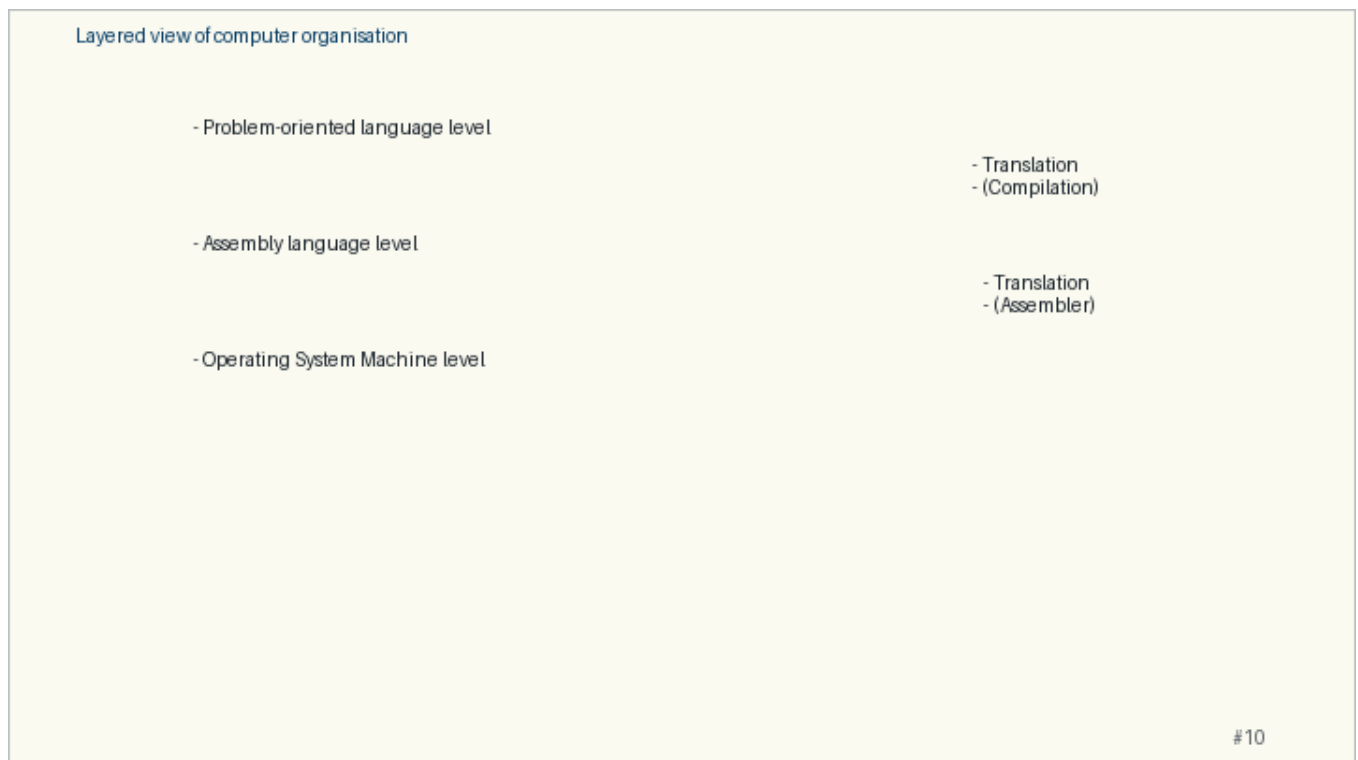
```
$ gcc -S foo.c
```

```
File Edit View Search Terminal Help
file "foo.c"
.section .rodata
.LC0:
.string "i+j=%d\n"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 0, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
subq $16, %rsp
movl $17, -12(%rbp)
movl $21, -8(%rbp)
movl -8(%rbp), %eax
movl -12(%rbp), %edx
addl %edx, %eax
movl %eax, -4(%rbp)
movl $.LC0, %eax
movl -4(%rbp), %edx
movl %edx, %esi
movq %rax, %rdi
movl $0, %eax
call printf
movl $0, %eax
leave
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE0:
.size main, .-main
.ident "GCC: (Ubuntu/Linaro 4.6.3-1ubuntu5) 4.6.3"
.section .note.GNU-stack,"",@progbits
1,2-11 All
```

1.15 Backup: Layered view of computer organisation

Original slide 10

- Problem-oriented language level
- L5
- Translation
- (Compilation)
- Assembly language level
- L4
- Translation
- (Assembler)
- Operating System Machine level
- L3



1.16 Backup: L3: Operating system

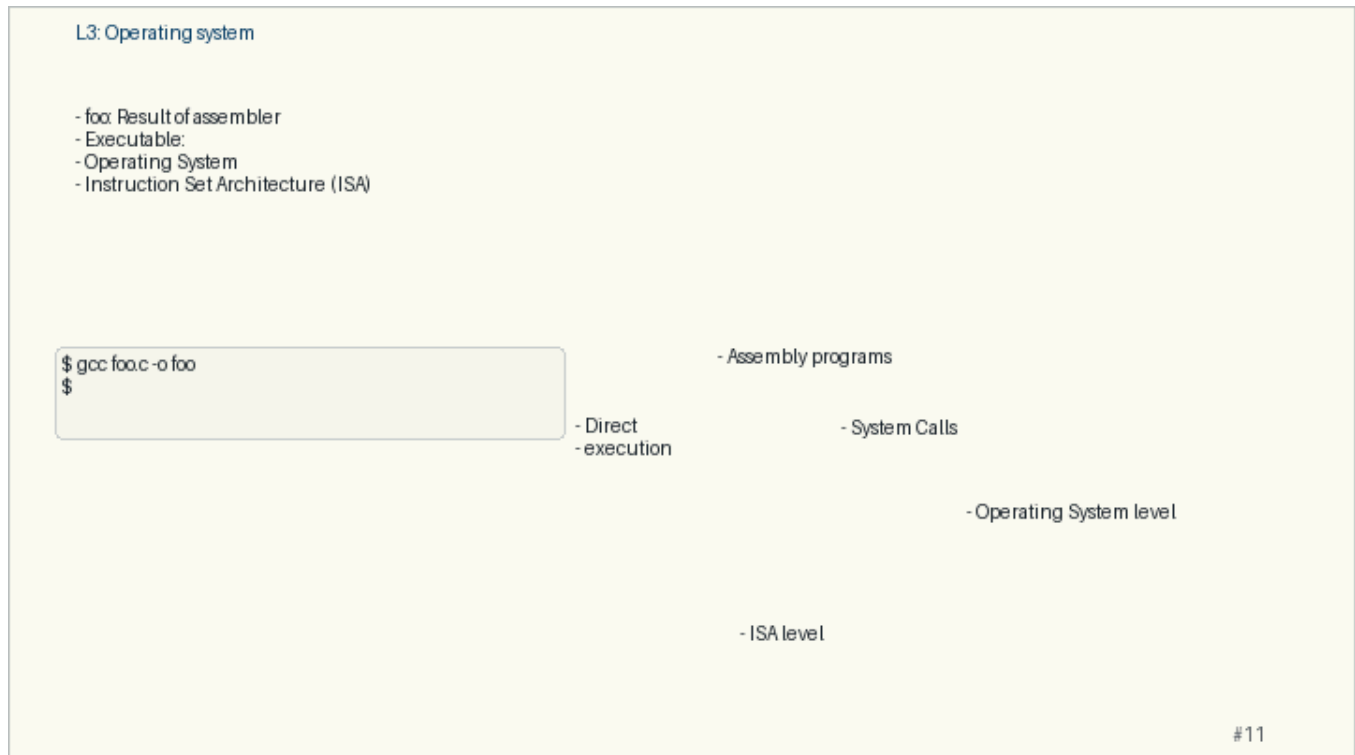
Original slide 11

- foo: Result of assembler
- Executable:
- Operating System
- Instruction Set Architecture (ISA)
- Assembly programs

```
$ gcc foo.c -o foo
$
```

- Direct
- execution
- System Calls

- Operating System level
- ISA level



1.17 Backup: Layered view of computer organisation

Original slide 12

- Problem-oriented language level
- L5
- Translation
- (Compilation)
- Assembly language level
- L4
- Translation
- (Assembler)
- Operating System Machine level
- L3
- Partial interpretation
- (O.S.)
- Instruction Set Architecture level
- L2

Layered view of computer organisation

- Problem-oriented language level

- Translation
- (Compiler)

- Assembly language level

- Translation
- (Assembler)

- Operating System Machine level

- Partial interpretation
- (O.S.)

- Instruction Set Architecture level

#12

1.18 Backup: L2: Instruction Set Architecture

Original slide 13

- foo:
- stored in the instruction memory
- consists of several machine code instructions
- depend on the ISA of the used processor (X86, ARM, SPARC, NIOS2...)
- Processor
- Data
- Instruction memory
- Adresses
- Results
- Instructions

L2: Instruction Set Architecture

- foo
- stored in the instruction memory
- consists of several machine code instructions
- depend on the ISA of the used processor (X86, ARM, SPARC, NIOS2...)

- Processor

- Data

- Instruction memory

- Adresses

- Results

#13

1.19 Backup: Layered view of computer organisation

Original slide 14

- Problem-oriented language level
- L5
- Translation
- (Compilation)
- Assembly language level
- L4
- Translation
- (Assembler)
- Operating System Machine level
- L3
- Partial interpretation
- (O.S.)
- Instruction Set Architecture level
- L2
- Interpretation
- (micro-program)
- or direct execution
- Microarchitecture level
- L1

Layered view of computer organisation

- Problem-oriented language level

- Translation
- (Compilation)

- Assembly language level

- Translation
- (Assembler)

- Operating System Machine level

- Partial interpretation
- (O.S.)

- Instruction Set Architecture level

- Interpretation
- (micro-program)
- or direct execution

- Microarchitecture level

#14

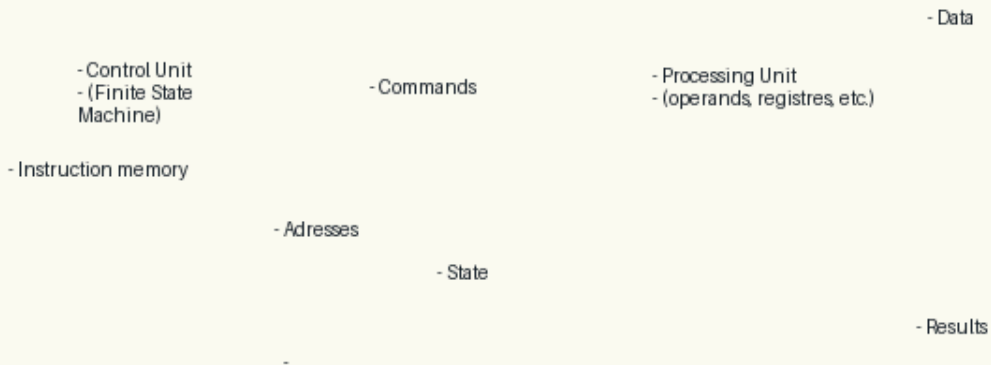
1.20 Backup: L1: Micro architecture

Original slide 15

- Each ISA instruction is implemented by a set of micro-operations (micro-ops)
- Micro-ops:
 - Created by the Control Unit
 - Describe how the Processing Unit has to behave to execute the ISA instruction
- Data
- Control Unit
 - (Finite State Machine)
- Processing Unit
 - (operands, registres, etc.)
- Commands
- Instruction memory
- Adresses
- State
- Results
- Instructions

L1: Microarchitecture

- Each ISA instruction is implemented by a set of micro-operations (micro-ops)
- Micro-ops
- Created by the Control Unit
- Describe how the Processing Unit has to behave to execute the ISA instruction



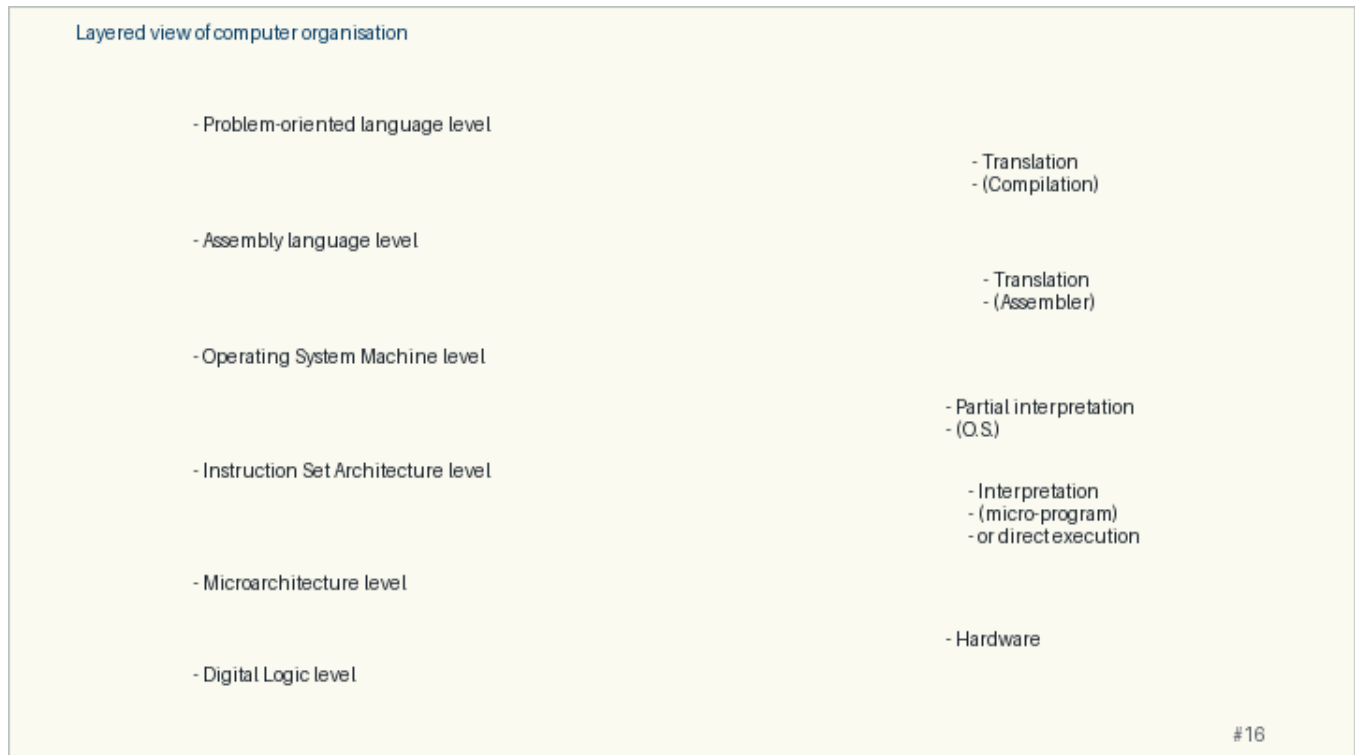
#15

1.21 Backup: Layered view of computer organisation

Original slide 16

- Problem-oriented language level
- L5
- Translation
- (Compilation)
- Assembly language level
- L4
- Translation
- (Assembler)
- Operating System Machine level
- L3
- Partial interpretation
- (O.S.)
- Instruction Set Architecture level
- L2
- Interpretation
- (micro-program)
- or direct execution
- Microarchitecture level
- L1
- Hardware

- Digital Logic level
- L0

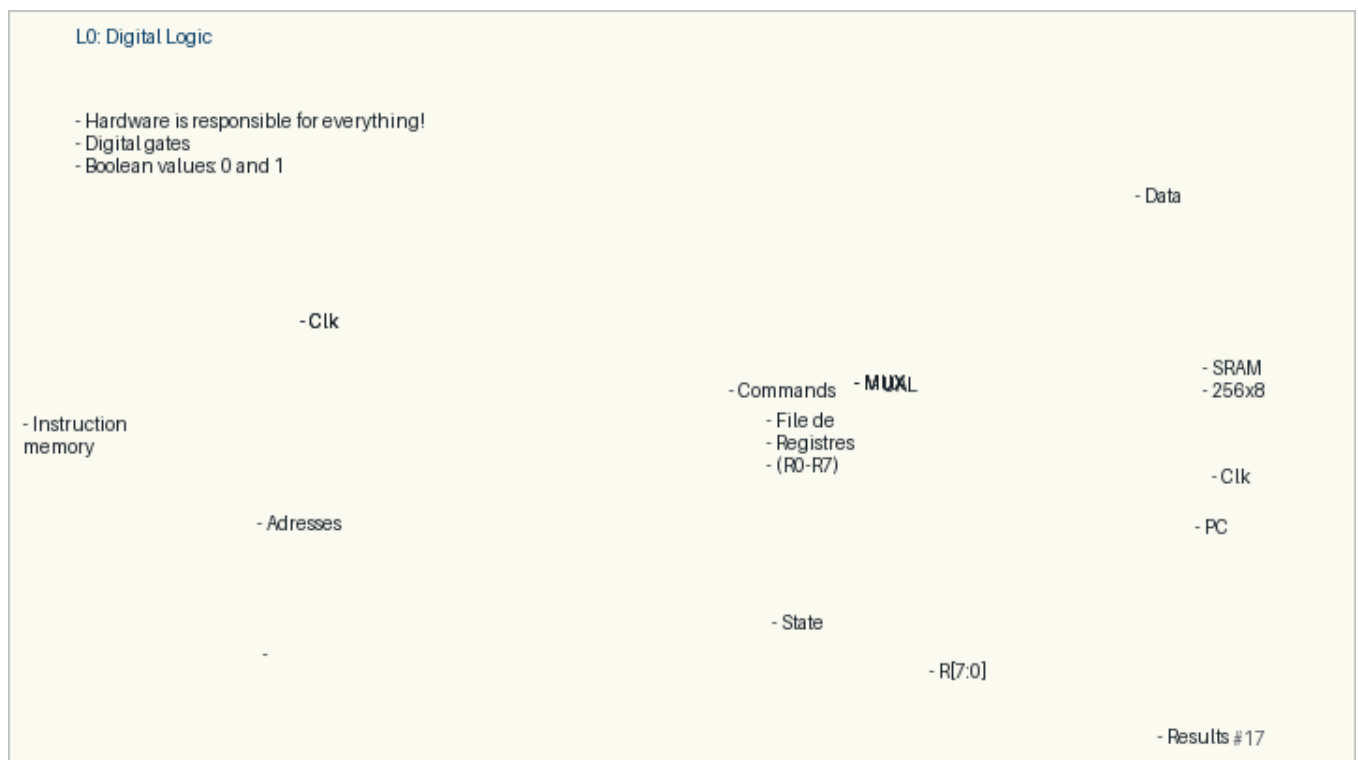


1.22 Backup: L0: Digital Logic

Original slide 17

- Hardware is responsible for everything!
- Digital gates
- Boolean values: 0 and 1
- Data
- Clk
- Clk
- 1
- 1
- SRAM
- 256x8
- MUX
- MUX
- MUX
- UAL
- 0
- 0
- Commands
- D0

- File de
- Registres
- (R0-R7)
- Instruction memory
- DQ
- A
- Clk
- Q0
- Q1
- Adresses
- PC
- 0
- 1
- A
- B
- State
- Instructions
- R
- R[7:0]
- Results

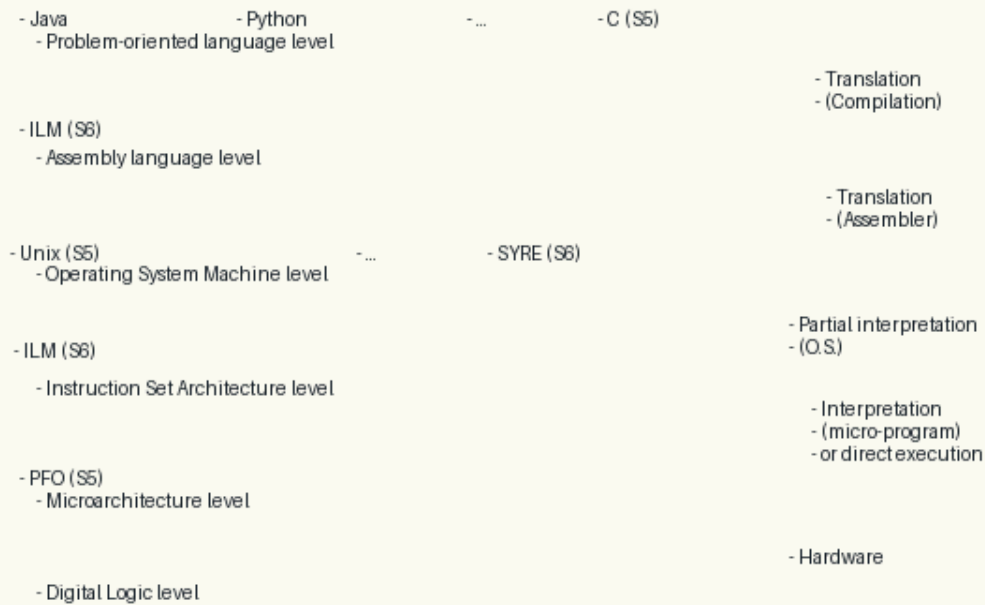


1.23 Backup: Layered view of computer organisation

Original slide 18

- Java
- Python
- ...
- C (S5)
- Problem-oriented language level
- L5
- Translation
- (Compilation)
- ILM (S6)
- Assembly language level
- L4
- Translation
- (Assembler)
- ...
- Unix (S5)
- SYRE (S6)
- Operating System Machine level
- L3
- Partial interpretation
- (O.S.)
- ILM (S6)
- Instruction Set Architecture level
- L2
- Interpretation
- (micro-program)
- or direct execution
- PFO (S5)
- Microarchitecture level
- L1
- Hardware
- Digital Logic level
- L0

Layered view of computer organisation



#18

1.24 Backup: Outline

Original slide 19

- Part 1: Linux and Shell scripting:
- 2 CM, 1 TD, 2 TP
- Part 2: C Language
- 9 CM, 2 TD, 10 TP
- Kahoot time ! (2 Q)
- Goto: <https://kahoot.it/>
- Insert: PIN number
- Insert: a nickname

Outline

- Part 1: Linux and Shell scripting:
 - 2 CM, 1 TD, 2 TP
- Part 2: C Language

- Kahoot time ! (2 Q)
- Goto: <https://kahoot.it/>
- Insert PIN number
- Insert a nickname

#19

2 Linux and OS

Live pacing: about 8 min

Question. What does the operating system do between the program and the machine?

Core claim. The OS gives a program controlled access to files, processes, memory, and devices.

Target capability. Students can place shell commands, processes, files, and permissions in the OS layer.

2.1 Main Path

- Start from the question: What does the operating system do between the program and the machine?
- Build the model: The OS gives a program controlled access to files, processes, memory, and devices.
- End with a checkable action: Students can place shell commands, processes, files, and permissions in the OS layer.
- Keep only this slide on the horizontal path during the live lecture.

- Part A: Linux and Shell scripting

#20

2.2 Worked Example

- Use this as the live trace: Compare `ls`, an editor, and a compiled program: all are processes asking the OS for services.
- Representative source slide: 23
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Some differences with Windows

There is no C:/, D:/ etc.
A common root /
Peripherals are subdirectories: /mnt/cdrom
User account
Private disk space: /private/student/5/25/2363415 or ~/2363415
Belongs to a group
Directories and files: Access rights
Interface system: Shell

#23

2.3 Anecdote Or Motivation

A useful mental image: user programs are guests; the OS is the host that owns the keys.

- Use this only if students need a reason to care.

- If the room is already following, skip down to the check question.

2.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Shell

Command Line Interface (CLI)

Input: The commands you want to execute (use the keyboard)

Output: Shell passes the commands to the OS, which executes them, and provides the result (usually to the screen)

Scripts: Combination of commands

Variables

Interaction with the Shell: Terminal emulators

Konsole, Xterm, Gnome-terminal...

Several shells are available

sh (Bourne shell), bash (Bourne shell), csh, zsh, ksh ...

#25

2.5 Backup Index

Original slides 20-25

- Part A: Linux and Shell scripting

#20

slide 20

Operating system

- Manages
- Process: creation, execution, termination, ...
- Memory: isolation, paging, virtual memory, ...
- Files: creation, access, permissions, ...
- Peripherals: usage, common interface, ...
- Provides a higher layer abstraction to interact with hardware peripherals
- System calls
- Focus: Utilisation of the Operating System
- Linux

#21

slide 21

Linux

- First prototype in 1991 (post to "comp.os.minix"):
- Linus Torvalds (Finland student - age 21):
- "Hello everybody out there using minix -
- I am doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386 (486) AT clones..."
- Naming:
- Linus: "Freax" -> "Free", "Freak", "x" (from Unix)
- Ari Lemmke (server admin): Renamed to "Linux"
- Free and open source operating system
- > 18.000.000 lines of source code

#22

slide 22

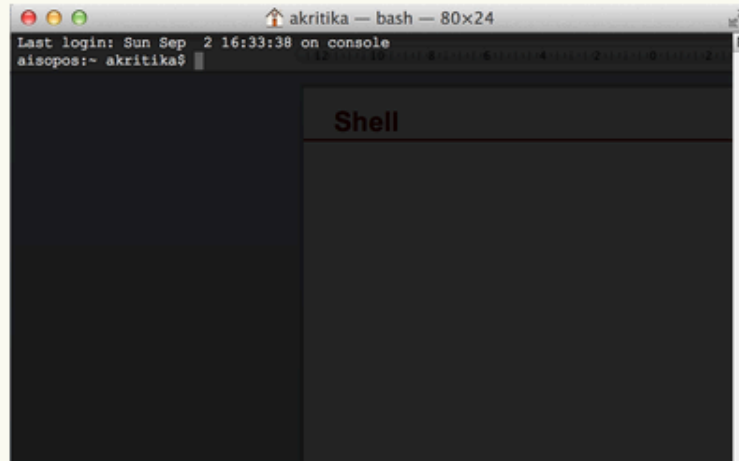
Some differences with Windows

There is no C:, D/ etc.
A common root /
Peripherals are subdirectories: /mnt/cdrom
User account
Private disk space: /private/student/5/25/2363415 or ~/2363415
Belongs to a group
Directories and files: Access rights
Interface system: Shell

#23

slide 23

Interface with the system: The Shell



#24

slide 24

Shell

Command Line Interface (CLI)

Input: The commands you want to execute (use the keyboard)

Output: Shell passes the commands to the OS, which executes them, and provides the result (usually to the screen)

Scripts: Combination of commands

Variables

Interaction with the Shell: Terminal emulators

Konsole, Xterm, Gnome-terminal...

Several shells are available

sh (Bourne shell), bash (Bourne shell), csh, zsh, ksh ...

#25

slide 25

Anchors: Part A: Linux and Shell scripting; Operating system; Linux; Some differences with Windows; Interface with the system: The Shell; Shell

2.6 Backup: Part A: Linux and Shell scripting

Original slide 20

- Part A: Linux and Shell scripting

#20

2.7 Backup: Operating system

Original slide 21

- Manages:
- Process: creation, execution, termination, ...

- Memory: isolation, paging, virtual memory, ...
- Files: creation, access, permissions, ...
- Peripherals: usage, common interface, ...
- Provides a higher layer abstraction to interact with hardware peripherals
- System calls
- Focus: Utilisation of the Operating System
- Linux

Operating system

- Manages
 - Process: creation, execution, termination, ...
 - Memory: isolation, paging, virtual memory, ...
 - Files: creation, access, permissions, ...
 - Peripherals: usage, common interface, ...
- Provides a higher layer abstraction to interact with hardware peripherals
- System calls
- Focus: Utilisation of the Operating System
- Linux

#21

2.8 Backup: Linux

Original slide 22

- First prototype in 1991 (post to "comp.os.minix"):
- Linus Torvalds (Finland student - age 21):
- "Hello everybody out there using minix -
- I am doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386 (486) AT clones..."
- Naming:
- Linus: "Freax" -> "Free", "Freak", "x" (from Unix)
- Ari Lemmke (server admin): Renamed to "Linux"
- Free and open source operating system
- > 18.000.000 lines of source code

Linux

- First prototype in 1991 (post to 'comp.os.minix');
- Linus Torvalds (Finland student - age 21):
- "Hello everybody out there using minix -
- I am doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386 (486) AT clones..."
- Naming:
- Linus: "Freax" -> "Free", "Freak", "x" (from Unix)
- Ari Lemmke (server admin): Renamed to "Linux"
- Free and open source operating system
- > 18.000.000 lines of source code

#22

2.9 Backup: Some differences with Windows

Original slide 23

There is no: C:/, D:/ etc.

A common root: /

Peripherals are subdirectories: /mnt/cdrom

User account:

Private disk space: /private/student/5/25/2363415 or ~/2363415

Belongs to a group

Directories and files: Access rights

Interface system: Shell

Some differences with Windows

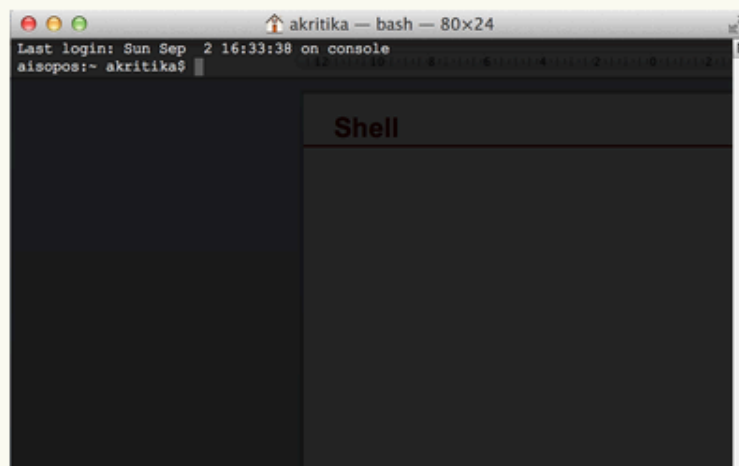
There is no C:, D:/ etc.
A common root /
Peripherals are subdirectories: /mnt/cdrom
User account
Private disk space: /private/student/5/25/2363415 or ~/2363415
Belongs to a group
Directories and files: Access rights
Interface system: Shell

#23

2.10 Backup: Interface with the system: The Shell

Original slide 24

Interface with the system: The Shell



#24

2.11 Backup: Shell

Original slide 25

Command Line Interface (CLI)

Input: The commands you want to execute (use the keyboard)

Output: Shell passes the commands to the OS, which executes them, and provides the result (usually to the screen)

Scripts: Combination of commands
Variables
Interaction with the Shell: Terminal emulators
Konsole, Xterm, Gnome-terminal...
Several shells are available
sh (Bourne shell), bash (Bourne shell), csh, zsh, ksh ...
Each user has a preselected shell
Selection in the file: /etc/passwd
Command to change the shell: chsh

Shell

Command Line Interface (CLI)
Input: The commands you want to execute (use the keyboard)
Output: Shell passes the commands to the OS, which executes them, and provides the result (usually to the screen)
Scripts: Combination of commands
Variables
Interaction with the Shell: Terminal emulators
Konsole, Xterm, Gnome-terminal...
Several shells are available
sh (Bourne shell), bash (Bourne shell), csh, zsh, ksh ...

#25

3 Directories and Paths

Live pacing: about 8 min

Question. How do commands find data and compose with other commands?

Core claim. Paths, permissions, streams, and redirections are the vocabulary of reproducible command-line work.

Target capability. Students can predict where a command reads, writes, searches, and why it may fail.

3.1 Main Path

- Start from the question: How do commands find data and compose with other commands?
- Build the model: Paths, permissions, streams, and redirections are the vocabulary of reproducible command-line work.
- End with a checkable action: Students can predict where a command reads, writes, searches, and why it may fail.
- Keep only this slide on the horizontal path during the live lecture.

Directories

- Home Directory (\$HOME)
- Each user has a home directory connected with his name
- Name: 2363415
- Home directory: /private/student/5/25/2363415 or ~/2363415
- Working Directory (\$PWD)
- Current directory
- When connected: Working directory = Home directory
- Current directory: .
- Parent directory: ..
- Root directory: /

#26

3.2 Worked Example

- Use this as the live trace: Start with `pwd`, `ls`, `find`, then redirect output into a file and inspect the result.
- Representative source slide: 27
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Paths

Absolute
From root directory /: /private/student/5/25/2363415
Relative
From working directory:
Assume the following working directory:
/private/student/5/25/2363415
You can access a file in another directory /private/student/5/25/2363414 by:
../2363414

#27

3.3 Anecdote Or Motivation

Most beginner Unix bugs are not algorithmic; they are wrong directory, wrong name, wrong rights, or wrong stream.

- Use this only if students need a reason to care.

- If the room is already following, skip down to the check question.

3.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Useful information

tab key:

One tab key: automatically fills the word you are started typing, if exists

Two tab keys: Prints all the existing words in that start with the letters that you have typed

clear command: Cleans the screen in the shell

Up arrow key: Shows the previous command

history command: Show all previous commands

!!: Executes the previous command

Do not use special characters in the file names, that include also the space character and the hyphen character (-).

Put as default language ENGLISH, the French translation is not complete ...

#30

3.5 Backup Index

Original slides 26-30

Directories

- Home Directory (\$HOME)

- Each user has a home directory connected with his name

- Name: 2363415

- Home directory: /private/student/5/25/2363415 or ~/2363415

- Working Directory (\$PWD)

- Current directory

- When connected: Working directory = Home directory

- Current directory: .

- Parent directory: ..

- Root directory: /

#26

slide 26

Paths

Absolute
From root directory `/:/private/student/5/25/2363415`
Relative
From working directory:
Assume the following working directory:
`/private/student/5/25/2363415`
You can access a file in another directory `/private/student/5/25/2363414` by:
`../2363414`

#27

slide 27

Wildcard characters

Special characters which can be used in combination with commands
One and only one character: `?`
Ex: `ls /etc/rc????`
From 0 to many characters: `*`
Ex: `ls /etc/rc*`
Specific characters: `[...]`
Ex: `ls [abc]oo.c`

#28

slide 28

How to treat them as normal (by the Shell)

Preserve the literal value of next character: \
Preserve the literal value of a character sequence: '...' (ticks)
As before, except for the characters \$ and `: "...` (quotes)
Command replacement: Execution of the command and replace with what it has been produced as output
`command` (back ticks)
\$(command)

#29

slide 29

Useful information

tab key:
One tab key: automatically fills the word you are started typing, if exists
Two tab keys: Prints all the existing words in that start with the letters that you have typed
clear command: Cleans the screen in the shell
Up arrow key: Shows the previous command
history command: Show all previous commands
!!: Executes the previous command
Do not use special characters in the file names, that include also the space character and the hyphen character (-).
Put as default language ENGLISH, the French translation is not complete ...

#30

slide 30

anchors: Directories; Paths; Wildcard characters; How to treat them as normal (by the Shell); Useful information

3.6 Backup: Directories

Original slide 26

- Home Directory (\$HOME)
- Each user has a home directory connected with his name
- Name: 2363415
- Home directory: /private/student/5/25/2363415 or ~/2363415
- Working Directory (\$PWD)

- Current directory
- When connected: Working directory = Home directory
- Current directory: .
- Parent directory: ..
- Root directory: /
- User home directory: ~

Directories

- Home Directory (\$HOME)
- Each user has a home directory connected with his name
- Name: 2363415
- Home directory: /private/student/5/25/2363415 or ~/2363415
- Working Directory (\$PWD)
- Current directory
- When connected: Working directory = Home directory
- Current directory: .
- Parent directory: ..
- Root directory: /

#26

3.7 Backup: Paths

Original slide 27

Absolute

From root directory / : /private/student/5/25/2363415

Relative

From working directory:

Assume the following working directory:

/private/student/5/25/2363415

You can access a file in another directory /private/student/5/25/2363414 by:

../2363414

Paths

Absolute
From root directory `/private/student/5/25/2363415`
Relative
From working directory:
Assume the following working directory:
`/private/student/5/25/2363415`
You can access a file in another directory `/private/student/5/25/2363414` by:
`../2363414`

#27

3.8 Backup: Wildcard characters

Original slide 28

Special characters which can be used in combination with commands

One and only one character: `?`

Ex: `ls /etc/rc????`

From 0 to many characters: `*`

Ex: `ls /etc/rc*`

Specific characters : `[...]`

Ex: `ls [abc]oo.c`

Wildcard characters

Special characters which can be used in combination with commands

One and only one character: `?`

Ex: `ls /etc/rc????`

From 0 to many characters: `*`

Ex: `ls /etc/rc*`

Specific characters: `[...]`

Ex: `ls [abc]oo.c`

#28

3.9 Backup: How to treat them as normal (by the Shell)

Original slide 29

```
Preserve the literal value of next character: \  
Preserve the literal value of a character sequence: '...' (ticks)  
As before, except for the characters $ and `: "... " (quotes)  
Command replacement: Execution of the command and replace with what it has been produced  
as output  
`command` (back ticks)  
$(command)
```

How to treat them as normal (by the Shell)

```
Preserve the literal value of next character: \  
Preserve the literal value of a character sequence: '...' (ticks)  
As before, except for the characters $ and `: "... " (quotes)  
Command replacement: Execution of the command and replace with what it has been produced as  
output  
`command` (back ticks)  
$(command)
```

#29

3.10 Backup: Useful information

Original slide 30

```
tab key:  
One tab key: automatically fills the word you are started typing, if exists  
Two tab keys: Prints all the existing words in that start with the letters that you have typed  
clear command: Cleans the screen in the shell  
Up arrow key: Shows the previous command  
history command: Show all previous commands  
!!: Executes the previous command  
Do not use special characters in the filenames, that include also the space character and the  
hyphen character (-).  
Put as default language ENGLISH, the French translation in not complete ...
```

Useful information

tab key:

One tab key: automatically fills the word you are started typing, if exists

Two tab keys: Prints all the existing words in that start with the letters that you have typed

clear command: Cleans the screen in the shell

Up arrow key: Shows the previous command

history command: Show all previous commands

!!: Executes the previous command

Do not use special characters in the file names, that include also the space character and the

hyphen character (-).

Put as default language ENGLISH, the French translation is not complete ...

#30

4 Commands and Files

Live pacing: about 8 min

Question. How do commands find data and compose with other commands?

Core claim. Paths, permissions, streams, and redirections are the vocabulary of reproducible command-line work.

Target capability. Students can predict where a command reads, writes, searches, and why it may fail.

4.1 Main Path

- Start from the question: How do commands find data and compose with other commands?
- Build the model: Paths, permissions, streams, and redirections are the vocabulary of reproducible command-line work.
- End with a checkable action: Students can predict where a command reads, writes, searches, and why it may fail.
- Keep only this slide on the horizontal path during the live lecture.

Structure of a command

- command arg1 arg2 ... argn

- The arguments of a command are the parameters passed during its call

The exit status is the value returned after the execution of the command to inform about normal or not execution

- command - argn - status

- The standard input is read from keyboard
-- Number 0 is stdin
-- Shortcut -

- std - in - std - out - The standard output is print to the screen
-- Number 1 is stdout

- stderr

- The standard error is by default printed to the screen and it is used by the commands to print the error messages
-- (Number 2 is stderr)

#31

4.2 Worked Example

- Use this as the live trace: Start with `pwd`, `ls`, `find`, then redirect output into a file and inspect the result.
- Representative source slide: 38
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Directory/File management Examples

- Kahoot time !
- (4 Q)

#38

4.3 Anecdote Or Motivation

Most beginner Unix bugs are not algorithmic; they are wrong directory, wrong name, wrong rights, or wrong stream.

- Use this only if students need a reason to care.

- If the room is already following, skip down to the check question.

4.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Directory/File management Examples

- Kahoot time !
- (4 Q)

#38

4.5 Backup Index

Original slides 31-38

Structure of a command

- command arg1 arg2 ... argn

- The arguments of a command are the parameters passed during its call

The exit status is the value returned after the execution of the command to inform about normal or not execution

- command - argn - status

- The standard input is read from keyboard
-- Number 0 is stdin
-- Shortcut -

- std
- in

- std
- out

- The standard output is print to the screen
-- Number 1 is stdout

- stderr

- The standard error is by default printed to the screen and it is used by the commands to print the error messages
-- (Number 2 is stderr)

#31

slide 31

Shell Commands

- Directory management
- File management
- Redirections
- Process management
- Shell programming: Scripts
- This course does NOT give an exhaustive list of commands and a complete command description ->
Use:
 - documentation
 - man or info command
 - apropos command

#32

slide 32

Directory management

Print the full pathname of the working directory: `pwd`
Change directories: `cd`
`cd` or `cd ~`: Go to home directory
`cd path`: Go to directory indicated by path
Creation/Delete directories
`mkdir dirname`: Creates a directory named `dirname`
`rmdir dirname`: Deletes the directory named `dirname` (empty)
`rm -r dirname`: Deletes the directory named `dirname` recursively (subdirectories) (not empty)
NO TRASHBIN!
Rename: `mv dirsource dirdest`
Hierarchical copy (subdirectories): `cp -r dirsource dirdest`

#33

slide 33

Listing

ls: Lists the files in the working directory
ls dirname: Lists the content of directory dirname
Some parameters passed as arguments:
-l: Use a long listing format (details)
-a: Not ignore entries starting with . (hidden folders/files)

#34

slide 34

File management

Create file by:
Update access & modification time: touch filename
Delete file: rm filename
Rename: mv filesource filedest
Copy:
cp filesource filedest
cp filesource dirdest
Remove non-directory suffix part from a filename: dirname
Remove the directory part suffix from a filename: basename
Count the number of words of a file: wc -w filename

#35

slide 35

Print the contents of a file:
 more filename: Prints page by page the content of file filename on the stdout (Q: Exit from more)
 less filename: Similar to more, but it allows to go up/down to the pages (Q: Exit from less)
 head -n 42 filename: Print the first 42 lines of filename
 tail -n 42 filename: Print the last 42 lines of filename
 Concatenate files (create and view files):
 cat filename1 ... filenameN (print context of files at stdout)
 cat: Print at stdout whatever is tapped at stdin (ctrl+D indicates EOF)
 -: Uses what is tapped with the keyboard instead of an existing file
 Difference between two files:
 diff filename1 filename2

slide 36

anchors: Structure of a command; Shell Commands; Directory management; Listing; File management; File management: Links

4.6 Backup: Structure of a command

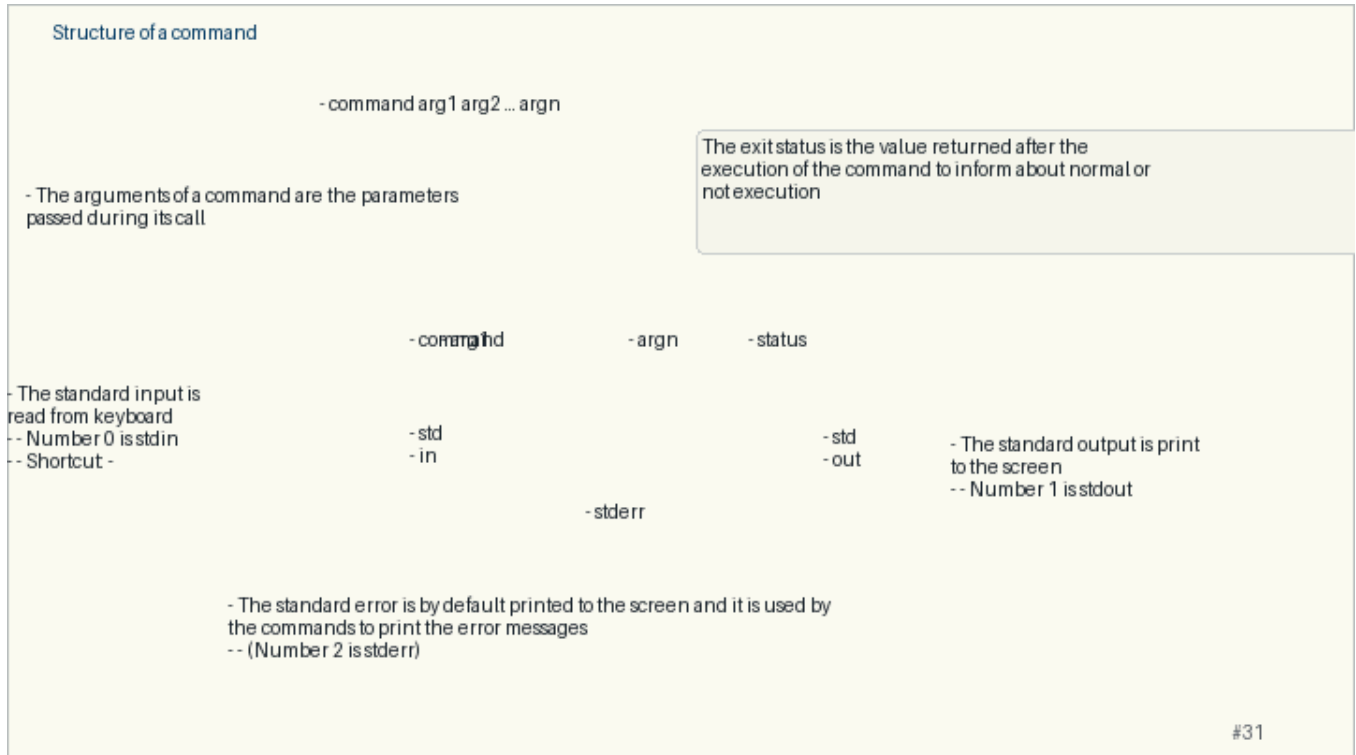
Original slide 31

- command arg1 arg2 ... argn

The exit status is the value returned after the execution of the command to inform about normal or not execution

- The arguments of a command are the parameters passed during its call
- command
- arg1
- argn
- status
- The standard input is read from keyboard
 - ▶ Number 0 is stdin
 - ▶ Shortcut: -
- std
- in
- std
- out
- The standard output is print to the screen
 - ▶ Number 1 is stdout
- stderr

- The standard error is by default printed to the screen and it is used by the commands to print the error messages
- ▶ (Number 2 is stderr)



4.7 Backup: Shell Commands

Original slide 32

- Directory management
- File management
- Redirections
- Process management
- Shell programming: Scripts
- This course does NOT give an exhaustive list of commands and a complete command description -> Use:
 - ▶ documentation
 - ▶ man or info command
 - ▶ apropos command

Shell Commands

- Directory management
- File management
- Redirections
- Process management
- Shell programming: Scripts
- This course does NOT give an exhaustive list of commands and a complete command description -> Use:
 - documentation
 - man or info command
 - apropos command

#32

4.8 Backup: Directory management

Original slide 33

```
Print the full pathname of the working directory: pwd
Change directories: cd
cd or cd ~: Go to home directory
cd path: Go to directory indicated by path
Creation/Delete directories
mkdir dirname: Creates a directory named dirname
rmdir dirname: Deletes the directory named dirname (empty)
rm -r dirname: Deletes the directory named dirname recursively (subdirectories) (not empty)
NO TRASHBIN !
Rename: mv dirsource dirdest
Hierarchical copy (subdirectories): cp -r dirsource dirdest
```

Directory management

```
Print the full pathname of the working directory: pwd
Change directories: cd
cd or cd ~: Go to home directory
cd path: Go to directory indicated by path
Creation/Delete directories
mkdir dirname: Creates a directory named dirname
rmdir dirname: Deletes the directory named dirname (empty)
rm -r dirname: Deletes the directory named dirname recursively (subdirectories) (not empty)
NO TRASHBIN!
Rename: mv dirsource dirdest
Hierarchical copy (subdirectories): cp -r dirsource dirdest
```

#33

4.9 Backup: Listing

Original slide 34

```
ls: Lists the files in the working directory
ls dirname: Lists the content of directory dirname
Some parameters passed as arguments:
-l: Use a long listing format (details)
-a: Not ignore entries starting with . (hidden folders/files)
```

Listing

```
ls: Lists the files in the working directory
ls dirname: Lists the content of directory dirname
Some parameters passed as arguments:
-l: Use a long listing format (details)
-a: Not ignore entries starting with . (hidden folders/files)
```

#34

4.10 Backup: File management

Original slide 35

```
Create file by:  
Update access & modification time: touch filename  
Delete file: rm filename  
Rename: mv filesource filedest  
Copy:  
cp filesource filedest  
cp filesource dirdest  
Remove non-directory suffix part from a filename: dirname  
Remove the directory part suffix from a filename: basename  
Count the number of words of a file: wc -w filename
```

File management

```
Create file by:  
Update access & modification time: touch filename  
Delete file: rm filename  
Rename: mv filesource filedest  
Copy:  
cp filesource filedest  
cp filesource dirdest  
Remove non-directory suffix part from a filename: dirname  
Remove the directory part suffix from a filename: basename  
Count the number of words of a file: wc -w filename
```

#35

4.11 Backup: File management

Original slide 36

```
Print the contents of a file:  
more filename: Prints page by page the content of file filename on the stdout (Q: Exit from more)  
less filename: Similar to more, but it allows to go up/down to the pages (Q: Exit from less)  
head -n 42 filename: Print the first 42 lines of filename  
tail -n 42 filename: Print the last 42 lines of filename  
Concatenate files (create and view files):  
cat filename1 ... filenameN (print context of files at stdout)  
cat: Print at stdout whatever is tapped at stdin (cntrl+D indicates EOF)  
- : Uses what is tapped with the keyboard instead of an existing file  
Difference between two files:  
diff filename1 filename2
```

Print the contents of a file:
more filename: Prints page by page the content of file filename on the stdout (Q: Exit from more)
less filename: Similar to more, but it allows to go up/down to the pages (Q: Exit from less)
head -n 42 filename: Print the first 42 lines of filename
tail -n 42 filename: Print the last 42 lines of filename
Concatenate files (create and view files):
cat filename1 ... filenameN (print context of files at stdout)
cat: Print at stdout whatever is tapped at stdin (ctrl+D indicates EOF)
-: Uses what is tapped with the keyboard instead of an existing file
Difference between two files:
diff filename1 filename2

4.12 Backup: File management: Links

Original slide 37

- Associate multiple filenames with a single file
- Hard: `ln filename1 filename2`
- New filename (filename2) to access existing file (filename1)
- Cannot link directories
- Cannot across file system boundaries
- Source of link is removed: Link always refers to the source
- Soft or Symbolic: `ln -s filename1 filename2`
- A symbolic path indicating the abstract location of another file
- Can link directories
- Can across file system boundaries
- Source of link is removed: Link not updated

File management Links

- Associate multiple filenames with a single file
- Hard: ln filename1 filename2
- New filename (filename2) to access existing file (filename1)
- Cannot link directories
- Cannot across file system boundaries
- Source of link is removed: Link always refers to the source
- Soft or Symbolic: ln -s filename1 filename2
- A symbolic path indicating the abstract location of another file
- Can link directories
- Can across file system boundaries

#37

4.13 Backup: Directory/File management: Examples

Original slide 38

- Kahoot time !
- (4 Q)

Directory/File management Examples

- Kahoot time !
- (4 Q)

#38

5 Permissions and Search

Live pacing: about 8 min

Question. How do commands find data and compose with other commands?

Core claim. Paths, permissions, streams, and redirections are the vocabulary of reproducible command-line work.

Target capability. Students can predict where a command reads, writes, searches, and why it may fail.

5.1 Main Path

- Start from the question: How do commands find data and compose with other commands?
- Build the model: Paths, permissions, streams, and redirections are the vocabulary of reproducible command-line work.
- End with a checkable action: Students can predict where a command reads, writes, searches, and why it may fail.
- Keep only this slide on the horizontal path during the live lecture.

Directory/File permissions

- rwx	- rwx	- rwx
- ---	- ---	- ---
- owned by user	- group	- others

Change permissions: chmod
chmod [who] op permissions [op permissions] * name

- Who:	- Op:	- Permissions:
- Owned by user (u)	- Add (+)	- Read or List (r)
- Group (g)	- Remove (-)	- Write (w)
- Others (o)	- Define (=)	- Execute or Access(x)
- Nothing (all)		

#39

5.2 Worked Example

- Use this as the live trace: Start with `pwd`, `ls`, `find`, then redirect output into a file and inspect the result.
- Representative source slide: 42
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

```

Search files based on criteria (print path):
find directory [criteria]
The search is performed to directory (recursive to subdirectories)
Apply a command at the found files: option -exec cmd {} \;
Apply the command cmd in each found file (it replaces the {})
End of the command: \;
Some search criteria:
-name filename: based on the filename (-iname: ignore case)
-user username: based on the user that owns the file
-size number[c|k|M]: based on size [B, KB, MB]
-type [d|f|l]: based on the type [directory, file, link]
-mtime x: based on time of modification (less than x days)
....
Criteria can be combined: -o -a -not

```

#42

5.3 Anecdote Or Motivation

Most beginner Unix bugs are not algorithmic; they are wrong directory, wrong name, wrong rights, or wrong stream.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

5.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

```

Print all lines of the files of the directory that contain a word: grep string directory
The string can be expressed as a regular expression
Whatever character: .
Repetition of pattern: *
Interval of characters: [x-y]
Return value: 0 found, else !=0
Examples:
grep -r test ./
grep 'm*t' filename
grep 'toto[1-3]' filename

```

#45

5.5 Backup Index

Original slides 39-45

Directory/File permissions

- rwx	- rwx	- rwx
- ---	- ---	- ---
- owned by user	- group	- others

Change permissions: chmod
chmod [who] op permissions [op permissions] * name

- Who:	- Op:	- Permissions:
- Owned by user (u)	- Add (+)	- Read or List (r)
- Group (g)	- Remove (-)	- Write (w)
- Others (o)	- Define (=)	- Execute or Access(x)
- Nothing (all)		

#39

slide 39

Directory/File permissions

- rwx	- rwx	- rwx
- ---	- ---	- ---
- owned by user	- group	- others

- Permissions can be given with numeric code per field
- In a field, you put 1 or 0 in each column to give or not the corresponding permission.

- 111	- 110	- 100
- owned by user	- group	- others

- 7	- 6	- 4
-----	-----	-----

chmod 764 filename

#40

slide 40

Directory/File permissions: Examples

- Kahoot time !
- (3 Q)

#41

slide 41

File management: Search files

Search files based on criteria (print path):
find directory [criteria]
The search is performed to directory (recursive to subdirectories)
Apply a command at the found files: option -exec cmd {} \
Apply the command cmd in each found file (it replaces the {})
End of the command: \
Some search criteria:
-name filename: based on the filename (-iname: ignore case)
-user username: based on the user that owns the file
-size number[c|k|M]: based on size [B, KB, MB]
-type [d|f|l]: based on the type [directory, file, link]
-mtime x: based on time of modification (less than x days)
....
Criteria can be combined: -o -a -not

#42

slide 42

- Kahoot time !
- (2 Q)

#43

slide 43

```
find /usr -name '*.c'  
find /usr -user angeliki  
find . -not \( -name '*.obj' -o -name '*.map' \)  
find . \( -user angeliki -a -name '*.c' \)  
find . -name '*.java' -exec lpr {} \;
```

#44

slide 44

Anchors: Directory/File permissions; Directory/File permissions: Examples; File management: Search files; File management: Search files: Examples; File management: Search in files

5.6 Backup: Directory/File permissions

Original slide 39

- r w x
- r w x
- r w x

- ---
- ---
- ---
- group
- others
- owned by user

Change permissions: chmod
 chmod [who] op permissions [,op permissions]* name

- Who:
- Owned by user (u)
- Group (g)
- Others (o)
- Nothing (all)
- Op:
- Add (+)
- Remove (-)
- Define (=)
- Permissions:
- Read or List (r)
- Write (w)
- Execute or Access(x)

Directory/File permissions

- rwx	- rwx	- rwx
- ---	- ---	- ---
- owned by user	- group	- others

Change permissions: chmod
 chmod [who] op permissions [,op permissions]* name

- Who:	- Op:	- Permissions:
- Owned by user (u)	- Add (+)	- Read or List (r)
- Group (g)	- Remove (-)	- Write (w)
- Others (o)	- Define (=)	- Execute or Access(x)
- Nothing (all)		

#39

5.7 Backup: Directory/File permissions

Original slide 40

- r w x
- r w x
- r w x
- ---
- ---
- ---
- group
- others
- owned by user
- Permissions can be given with numeric code per field
- In a field, you put 1 or 0 in each column to give or not the corresponding permission.
- 1 1 1
- 1 1 0
- 1 0 0
- group
- others
- owned by user

```
chmod 764 filename
```

- 7
- 6
- 4

Directory/File permissions

- rwx	- rwx	- rwx
- ---	- ---	- ---
- owned by user	- group	- others

- Permissions can be given with numeric code per field
 - In a field, you put 1 or 0 in each column to give or not the corresponding permission.

- 111	- 110	- 100
- owned by user	- group	- others

- 7	- 6	- 4	chmod 764 filename
-----	-----	-----	---

#40

5.8 Backup: Directory/File permissions: Examples

Original slide 41

- Kahoot time !
- (3 Q)

Directory/File permissions: Examples

- Kahoot time !
 - (3 Q)

#41

5.9 Backup: File management: Search files

Original slide 42

```
Search files based on criteria (print path):
find directory [criteria]
```

The search is performed to directory (recursive to subdirectories)
Apply a command at the found files: option `-exec cmd {} \;`
Apply the command `cmd` in each found file (it replaces the `{}`)
End of the command: `\;`
Some search criteria:
`-name filename`: based on the filename (`-iname`: ignore case)
`-user username`: based on the user that owns the file
`-size number[c|k|M]`: based on size [B, KB, MB]
`-type [d|f|l]`: based on the type [directory, file, link]
`-mtime x`: based on time of modification (less than x days)
....
Criteria can be combined: `-o -a -not`

File management Search files

Search files based on criteria (print path):
`find directory [criteria]`
The search is performed to directory (recursive to subdirectories)
Apply a command at the found files: option `-exec cmd {} \;`
Apply the command `cmd` in each found file (it replaces the `{}`)
End of the command: `\;`
Some search criteria:
`-name filename`: based on the filename (`-iname`: ignore case)
`-user username`: based on the user that owns the file
`-size number[c|k|M]`: based on size [B, KB, MB]
`-type [d|f|l]`: based on the type [directory, file, link]
`-mtime x`: based on time of modification (less than x days)
....
Criteria can be combined: `-o -a -not`

#42

5.10 Backup: File management: Search files

Original slide 43

- Kahoot time !
- (2 Q)

- Kahoot time !
- (2 Q)

#43

5.11 Backup: File management: Search files: Examples

Original slide 44

```
find /usr -name "*.c"
find /usr -user angeliki
find . -not \( -name "*.obj" -o -name "*.map" \)
find . \( -user angeliki -a -name "*.c" \)
find . -name "*.java" -exec lpr {} \;
```

```
find /usr -name "*.c"
find /usr -user angeliki
find . -not \( -name "*.obj" -o -name "*.map" \)
find . \( -user angeliki -a -name "*.c" \)
find . -name "*.java" -exec lpr {} \;
```

#44

5.12 Backup: File management: Search in files

Original slide 45

```
Print all lines of the files of the directory that contain a word: grep string directory
The string can be expressed as a regular expression
Whatever character : .
Repetition of pattern: *
Interval of characters: [x-y]
Return value: 0 found, else !=0
Examples:
grep -r test ./
grep "m*t" filename
grep "toto[1-3]" filename
```

File management: Search in files

```
Print all lines of the files of the directory that contain a word: grep string directory
The string can be expressed as a regular expression
Whatever character: .
Repetition of pattern: *
Interval of characters: [x-y]
Return value: 0 found, else !=0
Examples:
grep -r test ./
grep "m*t" filename
grep "toto[1-3]" filename
```

#45

6 Redirections

Live pacing: about 8 min

Question. How do commands find data and compose with other commands?

Core claim. Paths, permissions, streams, and redirections are the vocabulary of reproducible command-line work.

Target capability. Students can predict where a command reads, writes, searches, and why it may fail.

6.1 Main Path

- Start from the question: How do commands find data and compose with other commands?
- Build the model: Paths, permissions, streams, and redirections are the vocabulary of reproducible command-line work.
- End with a checkable action: Students can predict where a command reads, writes, searches, and why it may fail.
- Keep only this slide on the horizontal path during the live lecture.

Redirections

- Redirection of standard input (stdin):
- Use the content of a file as input to a command (instead of keyboard)
- command < filename
- Redirection of standard output (stdout)
- Print the output of a command into a file (instead of screen)
- command > filename (overwrite old content of file)
- command >> filename (append to the content of file)
- Pipe: redirect standard output to standard input of commands
- Use the output of a command as input to another command
- A subshell is created to execute command_2
- command_1 | command_2

#46

6.2 Worked Example

- Use this as the live trace: Start with `pwd`, `ls`, `find`, then redirect output into a file and inspect the result.
- Representative source slide: 47
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Redirections: Examples

- command
- command > res.txt
- command < data.txt
- Keyboard

- command
- std
- in

- std
- out

- Screen

- Keyboard

- command
- std
- in

- std
- out

- res.txt

- data.txt

- command
- std
- in

- std
- out

- Screen

#47

6.3 Anecdote Or Motivation

Most beginner Unix bugs are not algorithmic; they are wrong directory, wrong name, wrong rights, or wrong stream.

- Use this only if students need a reason to care.

- If the room is already following, skip down to the check question.

6.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

[Redirections](#) [Examples](#)

- Kahoot time !
- (3 Q)

#49

6.5 Backup Index

Original slides 46-49

[Redirections](#)

- Redirection of standard input (stdin):
- Use the content of a file as input to a command (instead of keyboard)
- command < filename
- Redirection of standard output (stdout)
- Print the output of a command into a file (instead of screen)
- command > filename (overwrite old content of file)
- command >> filename (append to the content of file)
- Pipe: redirect standard output to standard input of commands
- Use the output of a command as input to another command
- A subshell is created to execute command_2
- command_1 | command_2

#46

slide 46

Redirections: Examples

- command
- command > res.txt
- command < data.txt
- Keyboard

- command
- std
- in

- std
- out

- Screen

- Keyboard

- command
- std
- in

- std
- out

- res.txt

- data.txt

- command
- std
- in

- std
- out

- Screen

#47

slide 47

Redirections: Examples

- command < data.txt > res.txt
- com_1 < data.txt | com_2 > res.txt
- com_1 f1.txt < data.txt | com_2 b1 > res.txt

- res.txt

- data.txt

- com_1
- std
- in

- std
- out

- com_2
- std
- in

- std
- out

- res.txt

- data.txt

- f1.txt
- command
- std
- in
- arg1

- b1

- std
- out
- arg1

#48

slide 48

```
- Kahoot time !  
- (3 Q)
```

slide 49

Anchor: Redirections; Redirections: Examples

6.6 Backup: Redirections

Original slide 46

- Redirection of standard input (stdin):
- Use the content of a file as input to a command (instead of keyboard)
- `command < filename`
- Redirection of standard output (stdout)
- Print the output of a command into a file (instead of screen)
- `command > filename` (overwrite old content of file)
- `command >> filename` (append to the content of file)
- Pipe: redirect standard output to standard input of commands
- Use the output of a command as input to another command
- A subshell is created to execute `command_2`
- `command_1 | command_2`

Redirections

- Redirection of standard input (stdin):
- Use the content of a file as input to a command (instead of keyboard)
- command < filename
- Redirection of standard output (stdout)
- Print the output of a command into a file (instead of screen)
- command > filename (overwrite old content of file)
- command >> filename (append to the content of file)
- Pipe: redirect standard output to standard input of commands
- Use the output of a command as input to another command
- A subshell is created to execute command_2
- command_1 | command_2

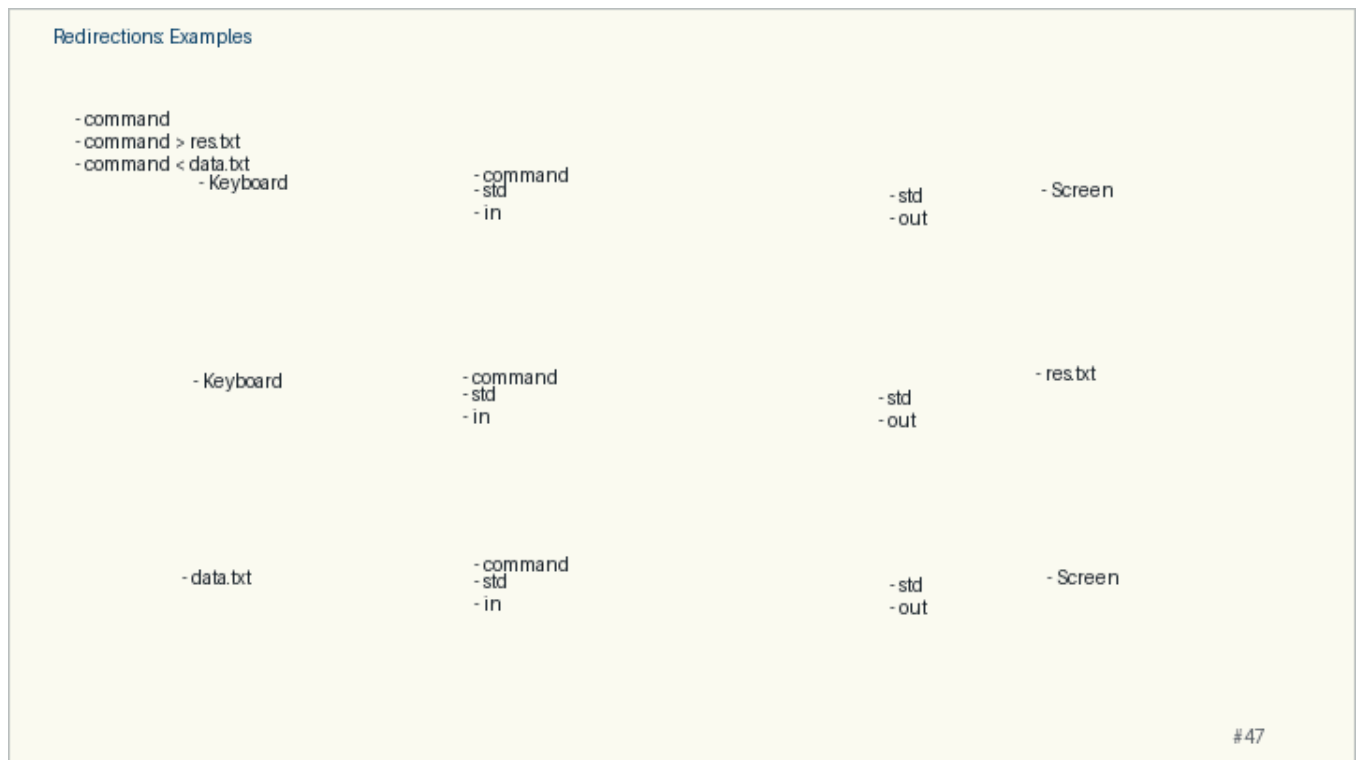
#46

6.7 Backup: Redirections: Examples

Original slide 47

- command
- command > res.txt
- command < data.txt
- command
- Keyboard
- std
- in
- Screen
- std
- out
- res.txt
- command
- Keyboard
- std
- in
- std
- out
- command
- data.txt
- Screen

- std
- in
- std
- out

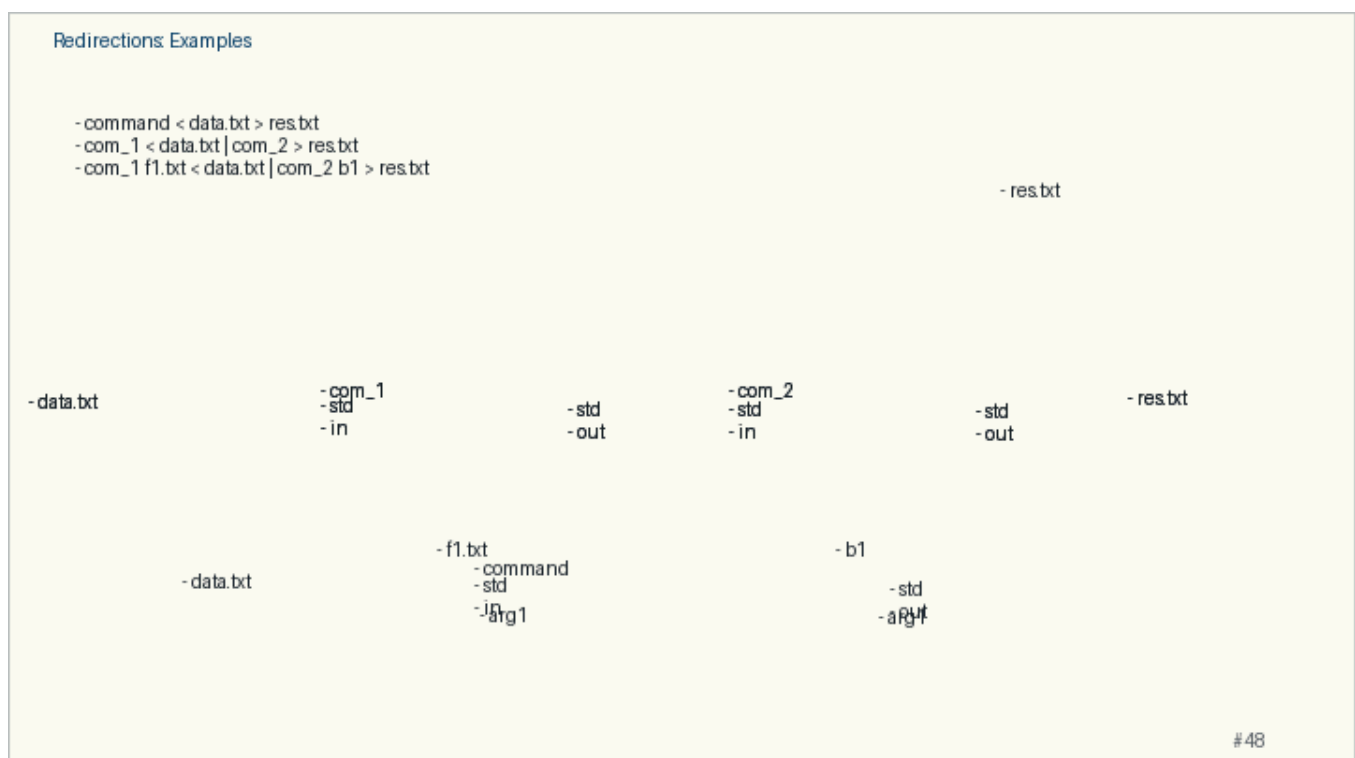


6.8 Backup: Redirections: Examples

Original slide 48

- command < data.txt > res.txt
- com_1 < data.txt | com_2 > res.txt
- com_1 f1.txt < data.txt | com_2 b1 > res.txt
- res.txt
- com_1
- com_1
- com_2
- com_2
- res.txt
- res.txt
- data.txt
- data.txt
- std
- in
- std
- in

- std
- out
- std
- out
- std
- in
- std
- in
- std
- out
- std
- out
- f1.txt
- b1
- command
- data.txt
- std
- in
- std
- out
- arg1
- arg1



6.9 Backup: Redirections: Examples

Original slide 49

- Kahoot time !
- (3 Q)

Redirections: Examples

- Kahoot time !
- (3 Q)

#49